

MoonBit Open Source Software Ecosystem Competition 2026

Project Declaration

1. Competition Direction

Basic Data Structures and Algorithms

2. Project Name

indexmap - Insertion-Order-Preserving Hash Map

3. Project Introduction

indexmap is an insertion-order-preserving hash map implemented for the MoonBit language. It combines the $O(1)$ average lookup efficiency of hash tables with the sequential access capability of arrays, while guaranteeing that iteration always maintains insertion order.

4. Project Features

4.1 Core Functionality

- $O(1)$ average time complexity for key-value lookup
- $O(1)$ time complexity for insertion-order index access
- Iteration always in insertion order
- Swap-remove semantics: removal uses swap-remove strategy, keeping internal array compact

4.2 Data Structure Design

Core design choices:

- Open addressing hash table: Uses linear probing for collision resolution, consistent with MoonBit standard library HashMap
- Separate order array: Maintains insertion order separately, decoupled from hash table
- Key storage strategy: Order array stores keys rather than indices, so rehash operations do not affect order

4.3 Implementation Details

```
struct Slot[K, V] {  
    hash : Int  
    key : K  
    mut value : V  
}  
  
struct IndexMap[K, V] {  
    mut entries : FixedArray[Slot[K, V]?]  
    order : Array[K]  
    mut capacity : Int  
    mut capacity_mask : Int  
    mut size : Int  
}
```

- Hash table: Uses FixedArray[Slot[K, V]?] to store key-value pairs
- Order array: Uses Array[K] to store keys in insertion order
- Capacity management: Auto-resize with load factor 0.5

5. API Design

5.1 Constructors

Function	Description
IndexMap::new()	Create empty map
IndexMap::with_capacity(n)	Create map with at least n slots
IndexMap::from(arr)	Create map from array

5.2 Core Operations

Method	Description
m.set(key, value)	Insert or update
m.get(key) -> V?	Lookup, returns Option
m.at(key) -> V	Lookup, panics if missing
m.contains_key(key)	Key existence check
m.remove(key) -> V?	Remove and return value

5.3 Index Access

Method	Description
m.get_index(i)	Key-value pair at position i
m.get_index_key(i)	Key at position i
m.get_index_value(i)	Value at position i

5.4 Iterators

Method	Description
m.iter()	Iterate key-value pairs in order
m.keys_iter()	Iterate keys in order
m.values_iter()	Iterate values in order

5.5 Operators

- m[key] - bracket read (panics if missing)
- m[key] = value - bracket write

6. Trait Implementation

6.1 Eq Trait

Supports equality comparison between two IndexMaps. Since implemented with hash table, comparison is unordered (maps with same content but different insertion order are equal).

6.2 Debug Trait

Supports debug output, displayed as IndexMap({key: value, ...}) format.

7. Test Coverage

The project includes 30 test cases covering:

- Basic CRUD operations (create, read, update, delete)
- Bracket operators (read and write)

- Insertion order preservation
- Swap-remove semantics
- Auto-resize (100 entries)
- Large-scale data correctness (1000 entries)
- String and integer keys
- Duplicate key handling
- Eq trait
- Empty map iteration

Test result: Total tests: 30, passed: 30, failed: 0.

8. Usage Example

```
test {
  // Create and populate
  let m = @indexmap.IndexMap::from([("b", 2), ("a", 1), ("c", 3)])

  // Key lookup - O(1)
  assert_eq(m.get("a"), Some(1))
  assert_eq(m["b"], 2) // panics if missing

  // Iteration preserves insertion order
  let keys = m.keys_iter().to_array() // ["b", "a", "c"]

  // Index access
  assert_eq(m.get_index_key(0), Some("b"))
  assert_eq(m.get_index(1), Some(("a", 1)))

  // Update (keeps original position)
  m.set("a", 99)
  // still: ["b", "a", "c"], but m["a"] == 99

  // Remove
  assert_eq(m.remove("a"), Some(99))
  // now: ["b", "c"]
}
```

9. Installation

```
moon add moonbit-community/indexmap
```

Then import in your .mbt files:

```
let m = @indexmap.IndexMap::new()
```

10. Project Structure

```
indexmap/
.github/workflows/ci.yml # GitHub Actions CI
indexmap.mbt             # Main implementation file
indexmap_test.mbt        # Test file
moon.mod.json            # Module metadata
moon.pkg                 # Package dependencies
README.md                # English documentation
```

DECLARATION.md	# Project declaration
LICENSE	# Apache-2.0 license

11. Technical Highlights

- Consistent design with standard library: Uses same open addressing + linear probing strategy as MoonBit standard library HashMap
- Efficient memory layout: Uses FixedArray for hash table, memory contiguous
- Smart resize strategy: Load factor 0.5, balancing space and performance
- Type safety: Complete generic support, compile-time type checking
- Idiomatic MoonBit syntax: Uses #alias for operator overloading, guard pattern matching, etc.

12. Future Extensions

- Entry API (entry(key).or_insert(default))
- Ordered iteration (sorted order)
- Serialization/deserialization support
- Thread-safe version
- More set operations (union, intersection, difference)

13. License

Apache-2.0

14. Contact

- Project URL: <https://gitlink.org.cn/yangayang/indexmap>
- Issue tracking: GitLink Issues

*This project participates in MoonBit Open Source Software Ecosystem Competition 2026
Direction: Basic Data Structures and Algorithms*